

ПРАКТИЧЕСКАЯ РАБОТА №1

Знакомство с языком Си. Введение. Структура программы.

Цель работы: изучить базовую структуру программы на языке Си. Освоить синтаксис функций стандартного ввода-вывода, научиться объявлять и инициализировать переменные, понять механизм создания и вызова пользовательских функций. Развить внимание к деталям и точности при работе с форматированием строк.

Общее требование: во всех заданиях используйте подключение заголовочного файла `<stdio.h>`, функцию `main()` как точку входа и завершайте программу оператором `return 0;`.

1.1. Памятная команда сборки.

Напишите программу, которая выводит две строки терминала: команду компиляции исходного файла `pw01-1.c` в исполняемый файл `pw01-1` и команду запуска полученного исполняемого файла. Используйте один вызов функции `printf()` и символ переноса строки.

Таблица 1.1.1 – Тестовые данные

Выходные данные
gcc pw01-1.c -o pw01-1 ./pw01

о 1.2. Необычная идентификация студента.

Напишите программу, выводящую ваши данные в четыре строки, используя ровно четыре оператора `printf()`, содержащие следующие строки:

- 1) Строка должна содержать ваши фамилию и инициалы, заключенные в квадратные скобки (например, [Иванов И.И.]).
- 2) Строка выводит только фамилию, выровненную по центру относительно длины скобок из первой строки.
- 3) Строка содержит только инициалы, выровненные по правому краю относительно длины скобок.
- 4) Строка представляет собой зеркальное отражение первой строки.

Таблица 1.2.1 – Тестовые данные

Выходные данные
[Иванов И.И.] Иванов И.И.]И.И. Иванов[

о 1.3. Передача данных по Tab'ам.

Напишите программу, передающую ваши имя, группу и наименование рабочего устройства (`hostname`). Выведите данные в две строки, используя всего один вызов `printf()`, а также вместо пробелов используйте символ табуляции. В конце каждой строки должна стоять контрольная сумма – точное количество символов в этой строке (включая буквы, пробелы и спецсимволы), заключенное в скобки.

Таблица 1.3.1 – Тестовые данные

Выходные данные			
Altenok	{8}		
GG-NNN	CSC00	{14}	

1.4. Хронометр эксплуатации.

Создайте целочисленные переменные `years`, `days_per_year` и `total_days`. Присвойте переменной `years` любое небольшое значение, переменной `days_per_year` значение 365. Вычислите значение переменной `total_days` как произведение количества лет на количество дней в году. Выведите исходные значения и результат вычисления в три строки.

Таблица 1.4.1 – Тестовые данные

Выходные данные	
YEARS = 3	
DAYS_PER_YEAR = 365	
TOTAL_DAYS = 1095	

1.5. Вектор состояния ядра.

Напишите программу, создающую целочисленную переменную `reactor_core` с заданным значением. Программа должна вычислить и вывести само значение, его удвоенное значение и его квадрат. Вывод должен быть оформлен как вектор состояния. Скобки и запятые с пробелами должны выводиться отдельными операторами `printf`. Запрещено выводить весь вектор одной строкой в одном `printf`.

Таблица 1.5.1 – Тестовые данные

Выходные данные	
[12, 24, 144]	

1.6. Жизнь, прошедшая в тиках.

Напишите программу, использующую переменную с заданным возрастом в годах и переводящую его в тики процессора. Используйте именованные константы (`#define` или `const`) для коэффициентов:

1 год = 365 дней, 1 день = 24 часа, 1 час = 3600 секунд

Выведите результат строго в обратном порядке и в одну строку, используя разделительный символ '|': сначала секунды, потом часы, потом дни, потом годы.

Таблица 1.6.1 – Тестовые данные

Выходные данные
Тики: 567648000 Часы: 157680 Дни: 6570 Годы: 18

1.7. Последовательность POST-диагностики.

Напишите программу, моделирующую загрузку модулей. Создайте функцию `load_mem()`, выводящую текст `MEM_OK`, и функцию `load_cpu()`, выводящую текст `CPU_OK`. В функции `main()` вызовите их так, чтобы итоговый вывод выглядел как единая непрерывная строка.

Внутри функций `load_mem()` и `load_cpu()` категорически запрещено использовать символ переноса строки `\n` или пробелы. Все разделители и переносы строк должны формироваться исключительно в теле `main()`.

Таблица 1.7.1 – Тестовые данные

Выходные данные
BOOT:MEM_OK CPU_OK:END

1.8. Сигнал синхронизации.

Определите функцию `pulse()`, выводящую на экран один символ. С помощью многократных вызовов функции `pulse()` в `main()` выведите лесенку из символов: в первой строке один символ, во второй строке два символа, в третьей строке три символа.

Внутри функции `pulse()` запрещено использовать перенос строки. Переносы строк должны осуществляться только с помощью дополнительных вызовов в теле `main()`.

Таблица 1.8.1 – Тестовые данные

Выходные данные
@ @@ @@@

1.9. Каскад инициализации модулей.

В языке Си функции могут вызывать друг друга. Напишите программу со следующей структурой. Функция `phase_1()` выводит слово ALPHA, затем вызывает функцию `phase_2()`, затем выводит слово GAMMA. Функция `phase_2()` выводит слово BETA. Функция `main()` выводит слово START, вызывает `phase_1()`, затем выводит слово END.

Таблица 1.9.1 – Тестовые данные

Выходные данные
START ALPHA BETA GAMMA END

о 1.10. Финальный вызов.

Напишите программу, моделирующую протокол аварийной передачи данных между двумя узлами сети. Определите символическую константу `NODE_ID`. Создайте переменную `packet_size`, равную произведению `NODE_ID` на 4, и переменную `total_transfer`, равную произведению `packet_size` на 3.

Определите функцию `ping()`, которая выводит текст PING без переноса строки. Определите функцию `pong()`, которая выводит текст PONG без переноса строки. Определите функцию `handshake()`, которая последовательно вызывает `ping()`, затем выводит дефис, затем вызывает `pong()`, затем выводит дефис и в конце вызывает `ping()`. Внутри функций `ping()`, `pong()` и `handshake()` запрещено использовать символ переноса строки и пробелы.

В функции `main()` сформируйте ровно три строки вывода. Первая строка содержит результат работы `handshake()`, двоеточие и значение `packet_size`. Вторая строка содержит результат работы `handshake()`, двоеточие и значение `total_transfer`. Третья строка содержит текст SESSION, двоеточие и текст CLOSED. Все переносы строк и двоеточия формируются исключительно в `main()`.

Таблица 1.10.1 – Тестовые данные

Выходные данные
PING-PONG-PING:168
PING-PONG-PING:504
SESSION:CLOSED